# Video 04 — Networking for behaviorist AI: tutorial companion

**Video file:** BlueImpulse_Video04_Networking_for_Behaviorist_AI.mp4
**Viewing position:** 4 of 5
**Duration:** 10:00 (600 seconds)
**Format:** MP4, 1920 × 1080, 60 fps
**File size:** 50,061,875 bytes (50.06 MB)

**Prerequisites:** Understand fixed reactive rules versus learned policies (Video 01 / Chapter 2), and latency versus bandwidth (Video 03 / Chapter 4).

**Learning goals:**

- Separate local control from shared services
- Interpret network messages safely
- Associate feedback with its original action

**Chapter and subtopic connections:**

- Chapter 5 — Applications / Networking for a behaviour-based agent
- Chapter 2 — Behaviorism & Behaviorist AI / React first: behaviour-based robots
- Chapter 6 — Strengths & Limitations / Put each computation where it fits

**Apply in the learning system:** Complete ACT-05 in the Study Guide / Markdown activity file. Use Test Bank questions TF17–TF20, MC17–MC20 and OE05 for review. The six-chapter PPT deck introduces the same chapter concepts through classroom discussion.

**Optional interactive companion:** ROVERLAB lets you apply these ideas by building a robot, connecting its network and comparing stimulus-response rules. Play at https://roverlab-blue-impulse.vercel.app and use the ROVERLAB activity in the Study Guide. It is a simplified rule-and-feedback model, not a neural network or the numerical Q-learning algorithm taught in Video 01.

**Materials:** Paper and a calculator are sufficient. The optional supporting Python demonstration uses Python 3.9 or newer with no third-party packages or account.

```
python3 supporting/BlueImpulse_Demonstrations.py --topic networked-ai
```

## How to learn with this tutorial

Watch the opening problem, write a prediction, and follow the labelled mechanism. Pause during each learning check before reading the feedback. After the worked example, change one input and explain why the result changes. The tutorial is self-contained: all inputs, instructions and expected outputs are below.

# Chapter timestamps and teaching route

| Time | Teaching step | Chapter / subtopic |
|---|---|---|
| 00:00–00:29 | Three commands. One decision. | Chapter 5 — Applications / Networking for a behaviour-based agent |
| 00:29–01:06 | Today's route | Chapter 5 — Applications / Networking for a behaviour-based agent |
| 01:06–01:59 | Reactive behaviour is not always learning | Chapter 2 — Behaviorism & Behaviorist AI / React first: behaviour-based robots |
| 01:59–02:45 | Keep the time-critical loop close | Chapter 5 — Applications / Networking for a behaviour-based agent |
| 02:45–03:36 | Let the network do useful shared work | Chapter 5 — Applications / Networking for a behaviour-based agent |
| 03:36–04:41 | A number needs context | Chapter 5 — Applications / Networking for a behaviour-based agent |
| 04:41–05:54 | Inspect three incoming messages | Chapter 5 — Applications / Networking for a behaviour-based agent |
| 05:54–06:43 | Match feedback to the original action | Chapter 5 — Applications / Networking for a behaviour-based agent |
| 06:43–07:31 | Define what an outage means | Chapter 6 — Strengths & Limitations / Put each computation where it fits |
| 07:31–08:26 | The feedback packet never arrived | Chapter 5 — Applications / Networking for a behaviour-based agent |
| 08:26–09:31 | Connect the responsibilities | Chapter 5 — Applications / Networking for a behaviour-based agent |
| 09:31–10:00 | Pip completes the shift | Chapter 5 — Applications / Networking for a behaviour-based agent |

## Reproduce the message decisions

The toy receiver expects device ID robot-7, units m, policy_version 3, age at most 200 ms, and a sequence greater than the last accepted sequence. Start at last accepted sequence 41. Assume authentication was checked upstream; a Boolean in this demonstration is not cryptography.

| Packet | Sequence | Age | Decision | Last accepted sequence after decision |
|---|---|---|---|---|
| A | 42 | 50 ms | ACCEPT | 42 |
| B | 43 | 600 ms | REJECT STALE | 42 |
| C | 42 | 60 ms | REJECT DUPLICATE | 42 |

Only accepted messages advance accepted-sequence state. Delivery, correct format and authenticity do not by themselves establish that a command is fresh and suitable.

**Practice:** Action 42 occurred but its reward packet never arrived. Is the observed reward zero? Should a repeated command automatically execute again?

**Checked answer:** No to both. Missing evidence is not an observed zero. Record missing data explicitly, associate any later feedback with its original transition, and follow the defined receiver policy for duplicates. Real designs also need authenticated identity, appropriate clock/session assumptions and a defined fallback.

## Guided explanation

### 00:00 — Three commands. One decision.

**Reference:** Chapter 5 — Applications / Networking for a behaviour-based agent

Three commands. Only one belongs now. Pip, our simulated campus delivery rover, receives a fresh command, an old command, and a command it has already handled. They look similar on screen. Treating them alike would be a mistake.

Have you ever wondered how a connected agent preserves the meaning of a message? Chapter five, Applications, connects local control, shared services, and message context.

### 01:06 — Reactive behaviour is not always learning

**Reference:** Chapter 2 — Behaviorism & Behaviorist AI / React first: behaviour-based robots

Start with Chapter two, Behaviorism and Behaviorist AI. A reactive agent may map a sensed condition directly to an action. A fixed rule such as avoiding a detected obstacle can produce useful behaviour without learning from reward. Reinforcement learning adds a different claim: experience is used to improve a policy.

A network connection does not turn a fixed rule into a learning algorithm, and not every connected agent learns online. For our campus delivery system example, think of a locally evaluated policy with separate, reviewed updates from a server. We will label architectural choices clearly instead of treating them as properties of every AI system.

### 01:59 — Keep the time-critical loop close

**Reference:** Chapter 5 — Applications / Networking for a behaviour-based agent

Time-critical sensing, policy evaluation, and actuator control can stay near the device. Our robot observes an obstacle, evaluates its current policy, and sends a control signal to the actuator hardware. This local path need not wait for every cloud exchange.

A real-time task must meet a specified deadline, not just be fast on average. Local processing removes some network dependency, but it still needs adequate compute, trustworthy sensing, and suitable physical controls. Local does not automatically mean safe. The requirements and failure behaviour still have to be designed and checked.

### 02:45 — Let the network do useful shared work

**Reference:** Chapter 5 — Applications / Networking for a behaviour-based agent

In our illustrative campus delivery system, each robot handles immediate motion control locally. A shared service can allocate tasks, coordinate inventory, and analyse selected telemetry from several devices. Imagine a local driver following a route while a dispatcher coordinates deliveries across the fleet.

The driver needs immediate awareness; the dispatcher benefits from a broader view. The analogy separates responsibilities. The network may distribute reviewed policy updates, but a received update should not automatically bypass validation. Edge and cloud are not opposing teams. Put each part of the workload where its deadlines and resource needs fit.

### 03:36 — A number needs context

**Reference:** Chapter 5 — Applications / Networking for a behaviour-based agent

A message needs enough context to be interpreted correctly. A number by itself is not a complete instruction. Device identity tells us which source or destination it concerns. A timestamp helps determine freshness. A sequence number can expose repeated or out-of-order messages. Units prevent confusing metres with centimetres.

A policy version connects an action or update to the model that produced it. These fields are useful evidence, but fields alone do not authenticate a sender. Security checks must support the claimed identity. Before acting, the receiver needs a policy for rejecting stale, unexpected, or unauthorised commands.

Pip is a story name; robot-seven is the device identity used in this demonstration. A receiver must check the identity expected by the protocol, not guess from a friendly label.

### 04:41 — Inspect three incoming messages

**Reference:** Chapter 5 — Applications / Networking for a behaviour-based agent

Run the supporting packet-checking example. This is an illustrative receiver design, not a universal network protocol. Our toy receiver expects robot seven, metres as the unit, and policy version three. For this demonstration, it rejects commands older than two hundred milliseconds and sequences already processed.

The first message has the expected context, a new sequence, and an age of fifty milliseconds. It passes these checks. The second is six hundred milliseconds old. The receiver rejects it as stale, even if the network eventually delivered it. The third repeats an already processed sequence. The

receiver rejects it as a duplicate instead of executing the command twice.

Our example assumes authentication was checked elsewhere. It does not implement cryptographic authentication or certify a safety-critical controller. The useful lesson is why delivery alone is insufficient: the receiver must decide whether the message is still meaningful.

## 05:54 — Match feedback to the original action

**Reference:** Chapter 5 — Applications / Networking for a behaviour-based agent

Delayed feedback must be associated with the action that produced it. Suppose action forty-two turns left, and action forty-three moves forward. Their feedback packets arrive in the opposite order. Updating whichever action happened most recently would connect evidence to the wrong decision.

Use identifiers and recorded context to match each result with its original state and action before a learning update. If a feedback packet is lost, the evidence is missing. That does not silently mean the reward was zero. This is how networking details can affect the data used by a learning algorithm, even when policy evaluation stays local.

## 06:43 — Define what an outage means

**Reference:** Chapter 6 — Strengths & Limitations / Put each computation where it fits

Now inspect the outage condition for our illustrative delivery system. The design must say whether the robot pauses, completes a limited task, or enters another defined state. Continuing safely during a link outage cannot simply be assumed because some computation is local.

Consider expiring permissions, missing coordination, conflicting routes, and how the system reconnects after the interruption. Record what happened locally so the shared service can reconcile state when communication returns. The right fallback depends on the task and its hazards. Our tutorial demonstrates the reasoning rather than certifying a physical robot.

## 07:31 — The feedback packet never arrived

**Reference:** Chapter 5 — Applications / Networking for a behaviour-based agent

Pause and think. The robot performed action forty-two, but its reward packet never arrived. Should the learner record a reward of zero? And should a repeated command be executed again automatically? No to both. Missing evidence is not an observed zero, and a duplicate needs the receiver's defined handling policy.

Associate any later feedback with the original action. Decide how long to wait and how to record missing data. Then explain why a timestamp alone does not prove a message came from an authorised sender. Use the packet demonstration to change an age or sequence and predict which validation rule should reject it.

**Check your answer:** No. Missing evidence is not an observed zero.

## 08:26 — Connect the responsibilities

**Reference:** Chapter 5 — Applications / Networking for a behaviour-based agent

Keep suitable time-critical work local, and use the network for telemetry, coordination, and reviewed updates. Messages need interpretable context and appropriate authentication. Stale or repeated commands must not be treated as automatically valid. Delayed learning feedback belongs to its original action. Lost evidence must not silently become zero reward.

Decide which responsibility belongs on the robot, at a nearby gateway, or in the cloud. Finally, distinguish a fixed reactive rule, a deployed learned policy, and an algorithm currently updating that policy. Draw the local and shared responsibilities, then explain your outage behaviour before calling the architecture complete.

A message can be perfectly formatted and still be too late, duplicated, or unauthorised. Meaning depends on the receiver's state and the surrounding checks.

## Finish by explaining, not copying

Complete ACT-05 and the named Test Bank questions, then explain the mechanism without following the highlight. State one assumption of the worked model and one condition that could make the outcome differ in a real system. A correct calculation supports only the conclusion it actually measures.